

Основы глубинного обучения

Лекция 4

Оптимизация в глубинном обучении

Евгений Соколов

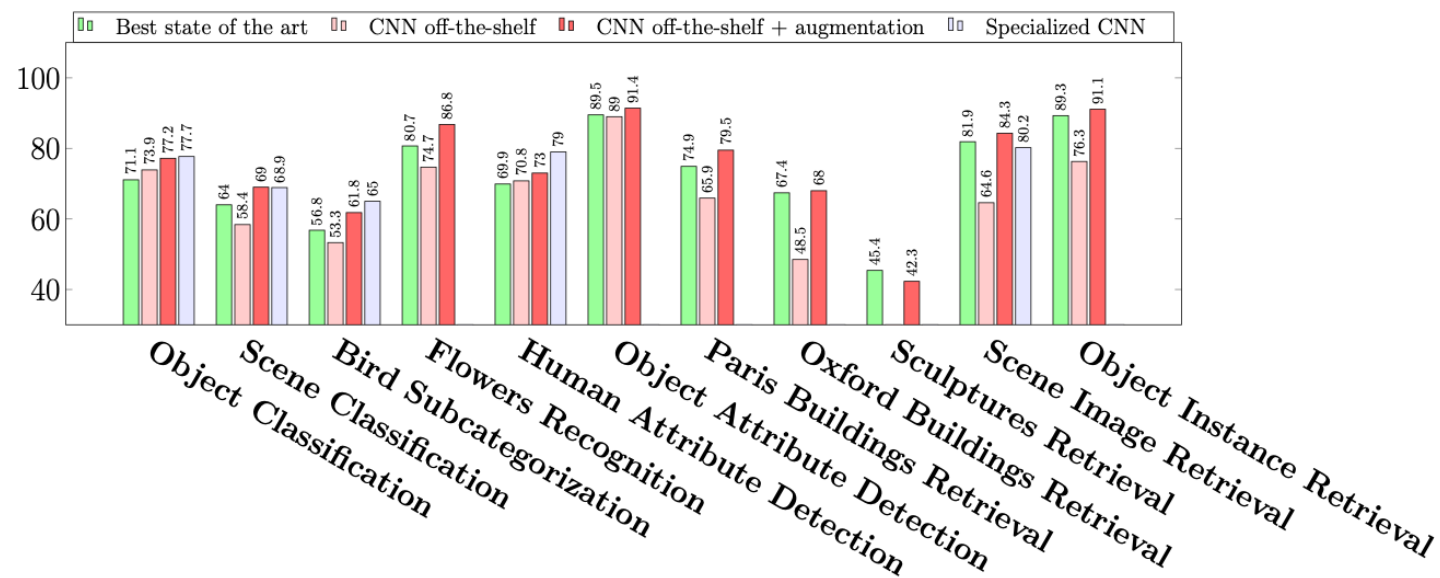
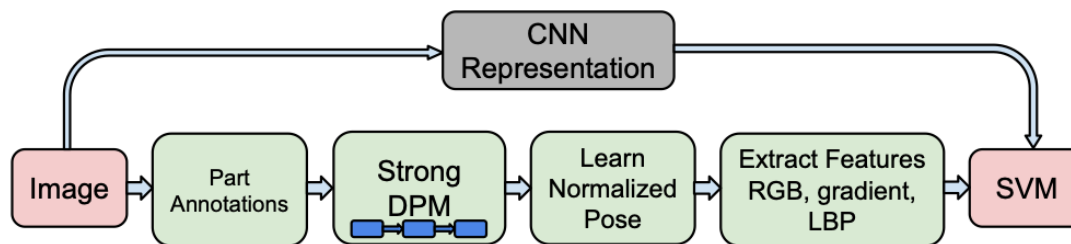
esokolov@hse.ru

НИУ ВШЭ, 2026

Представления с последних слоёв

- Важное наблюдение: выходы последних слоёв являются хорошими признаковыми описаниями изображений
- Полезны во многих задачах
- Например, поиск похожих изображений

Представления с последних слоёв



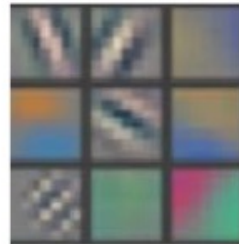
Сейчас: кодировщики изображений

- Сеть учат ради хороших признаков на огромных данных
- CLIP: картинки вместе с подписями, 400 млн пар
 - задача: угадать, какая подпись к какой картинке относится (в батче из N пар найти N правильных среди N^2 сочетаний)
 - признаки картинки и текста попадают в одно пространство → поиск картинок по тексту
- DINOv2 / DINOv3: вообще без разметки
 - задача: по маленькому кусочку картинки предсказать то, что сеть-«учитель» выдаёт по всей картинке
 - DINOv3 — 1,7 млрд изображений, 7 млрд параметров

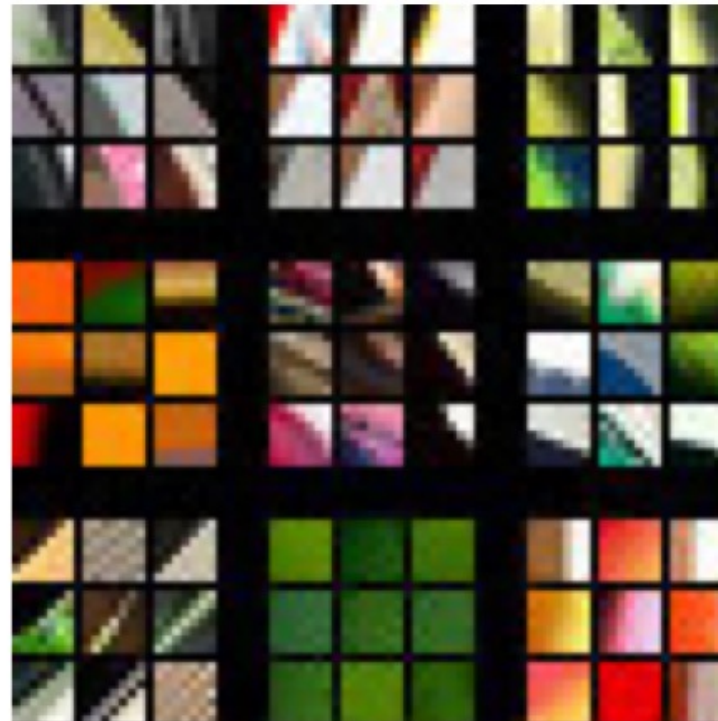
Представления с последних слоёв

- Не интерпретируется (в отличие от классического компьютерного зрения)
- По смыслу — индикаторы наличия каких-то паттернов

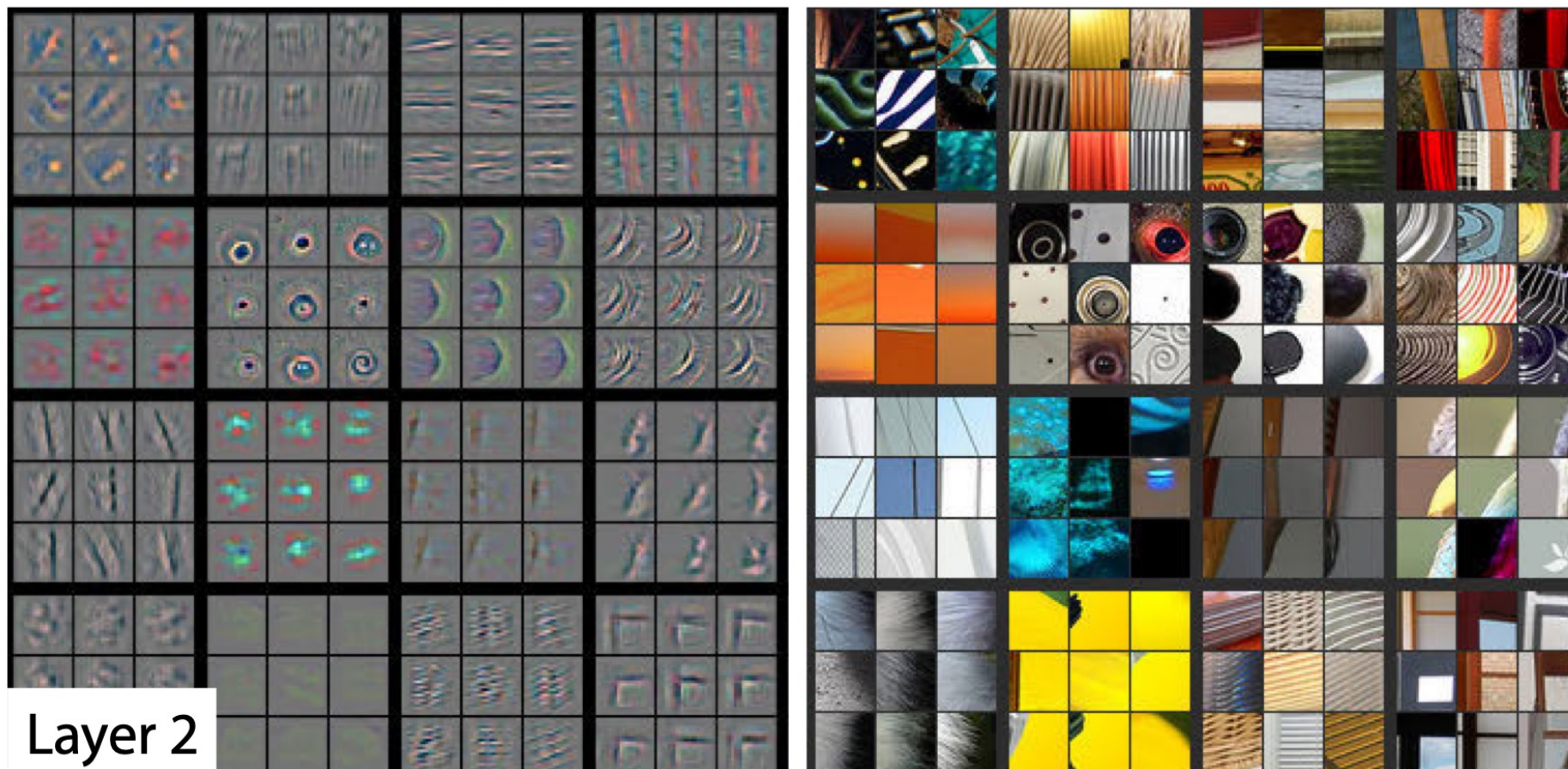
Представления с разных слоёв



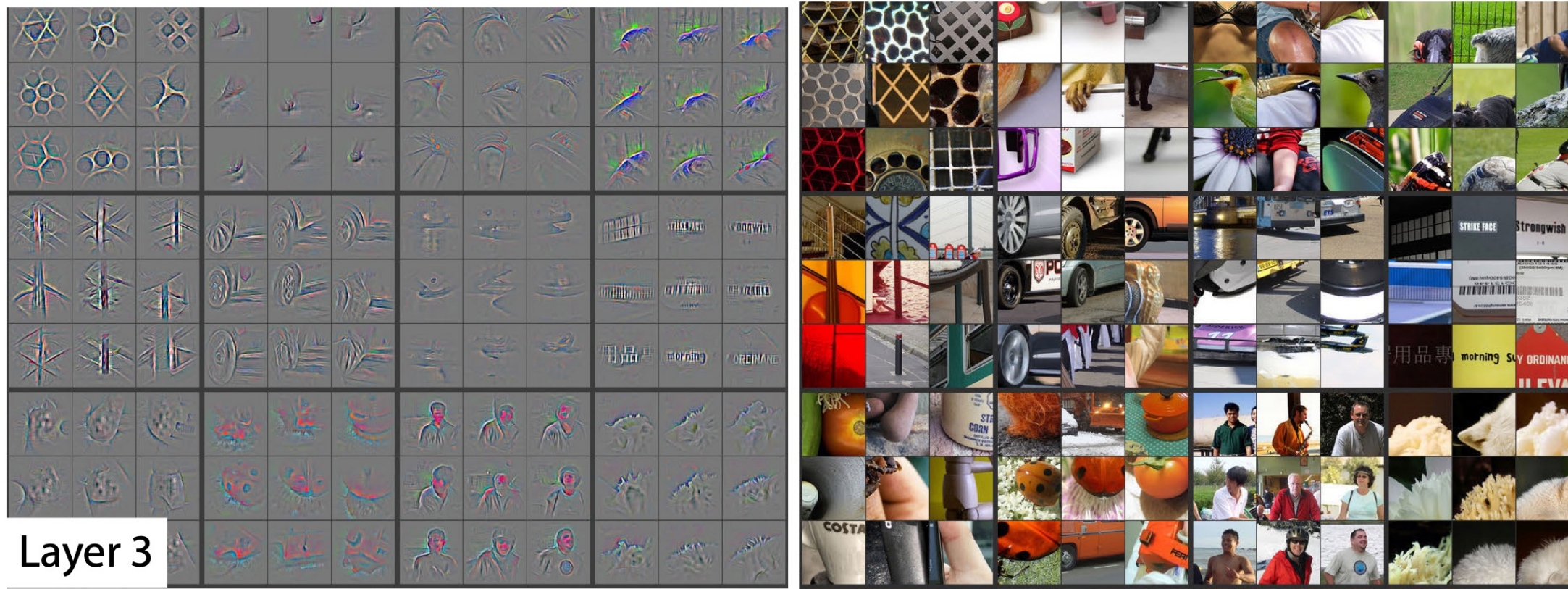
Layer 1



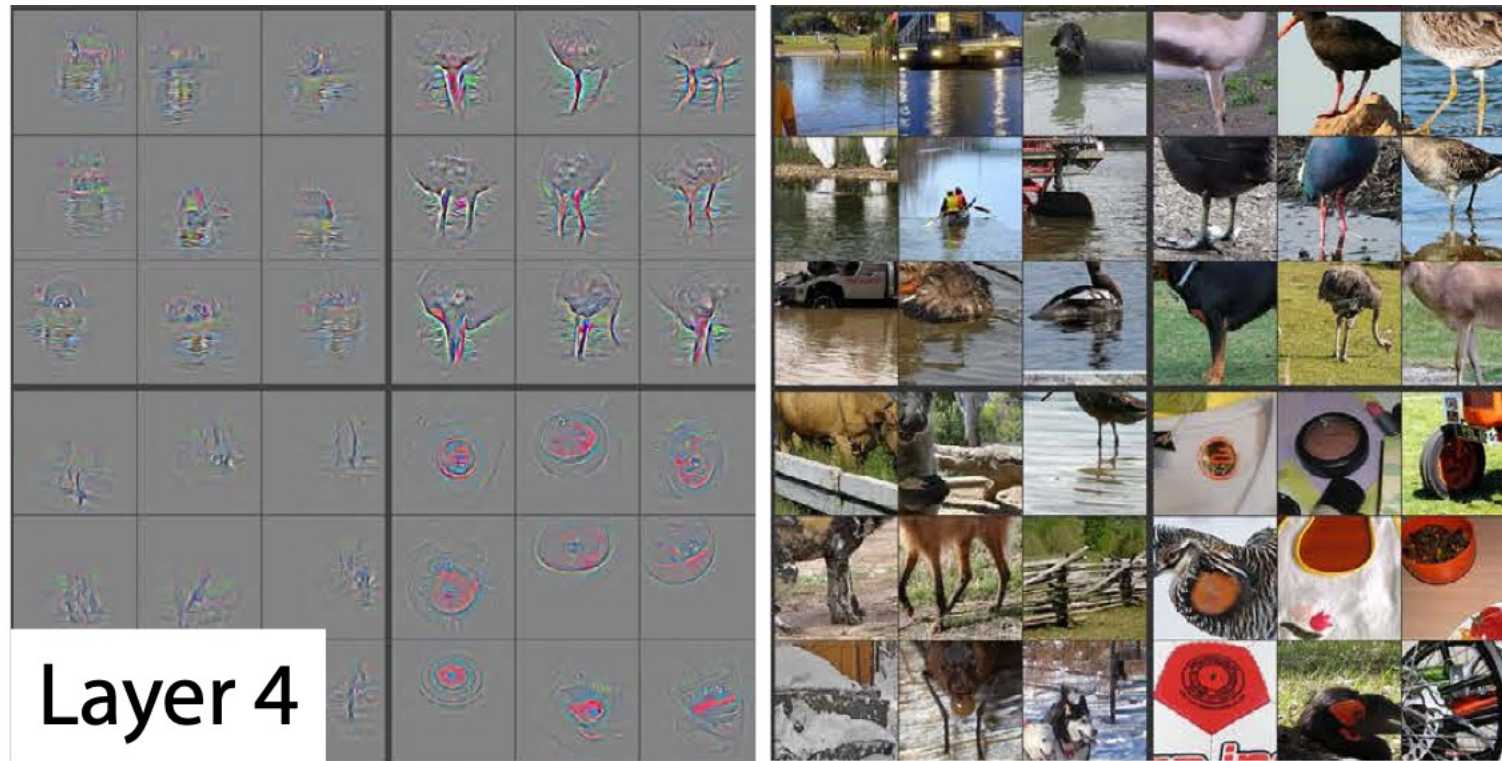
Представления с разных слоёв



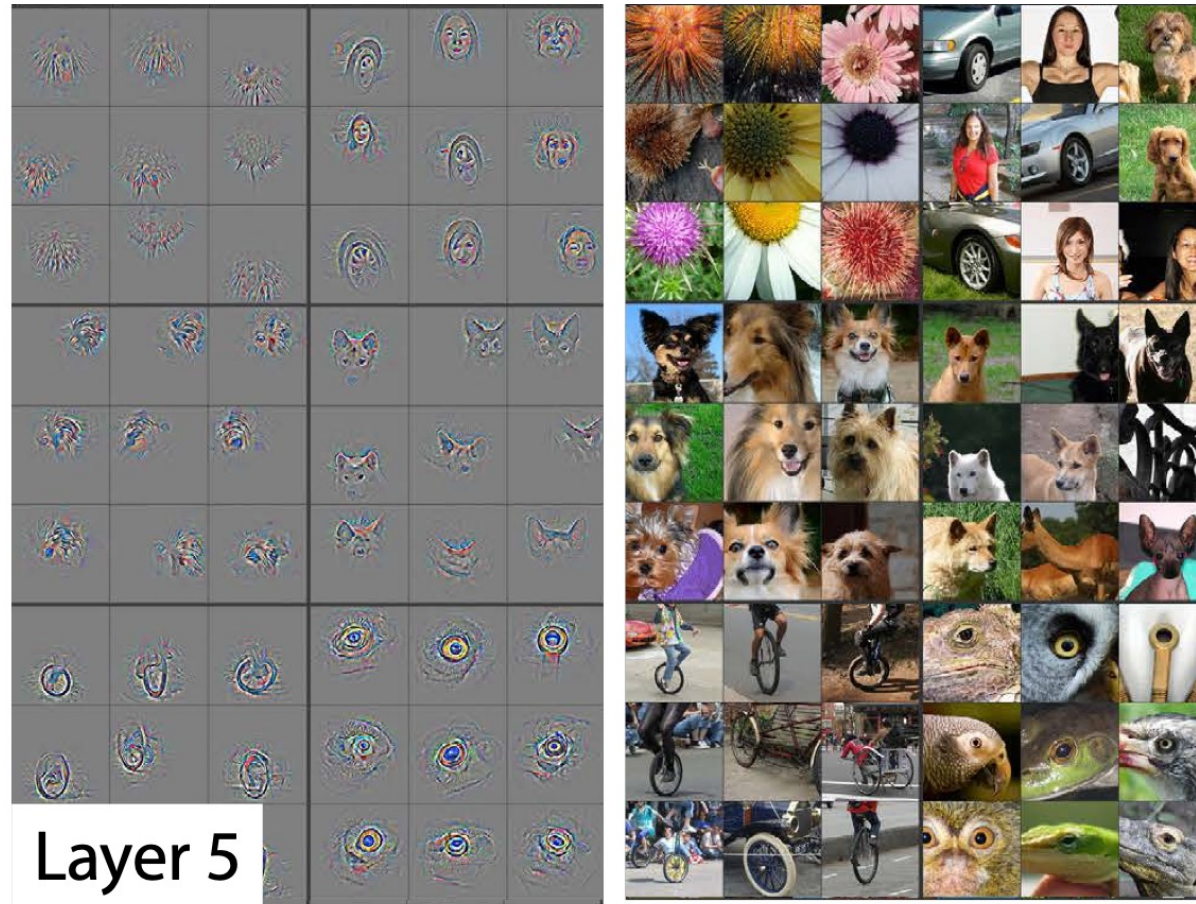
Представления с разных слоёв



Представления с разных слоёв



Представления с разных слоёв



Стохастический градиентный спуск

Градиентный спуск

1. Начальное приближение: w^0

2. Повторять:

$$w^t = w^{t-1} - \eta \nabla Q(w^{t-1})$$

3. Останавливаемся, если ошибка на валидационной выборке перестаёт убывать

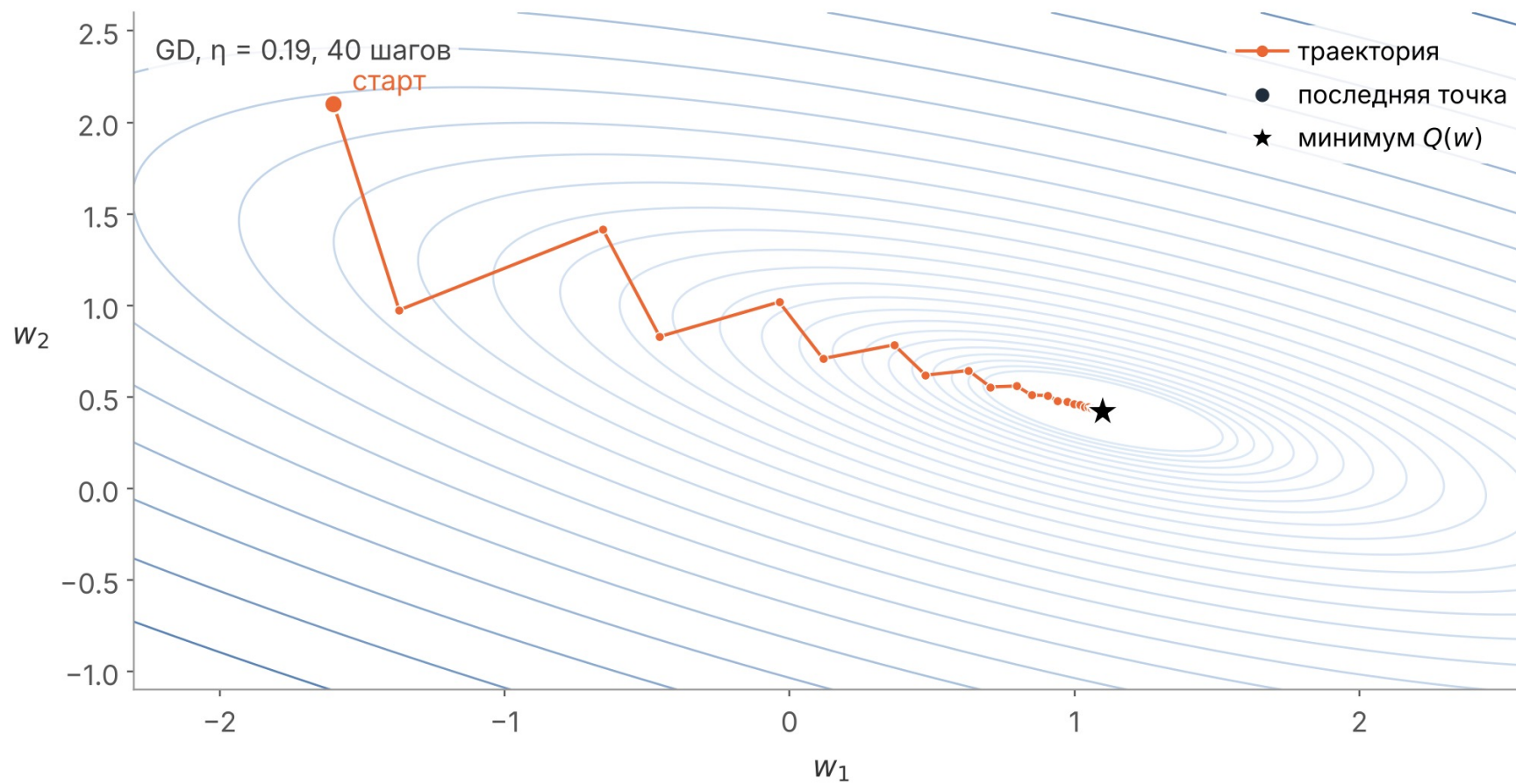
Стохастический градиентный спуск

1. Начальное приближение: w^0
2. Повторять, каждый раз выбирая случайный объект i_t :

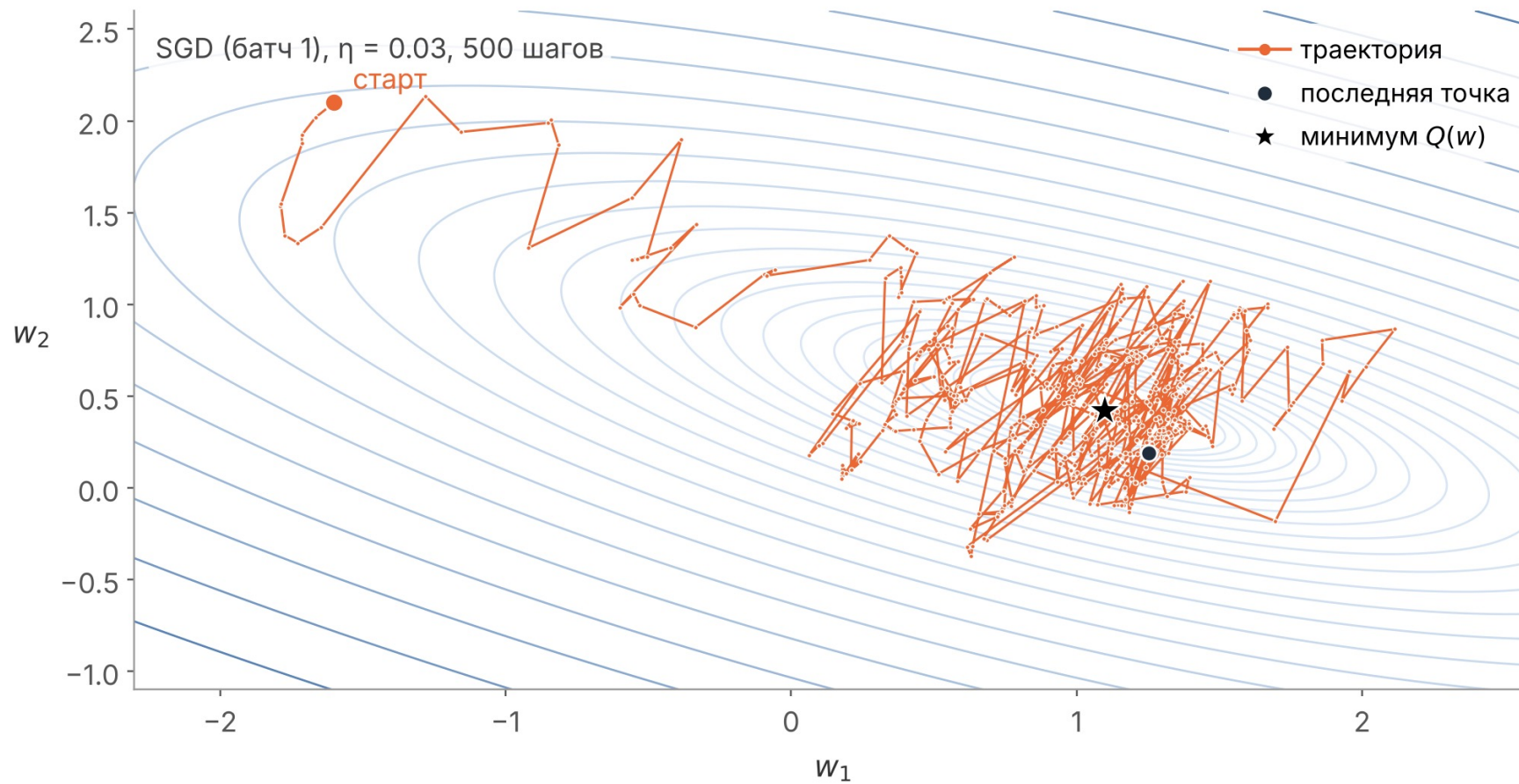
$$w^t = w^{t-1} - \eta \nabla L(y_{i_t}, a(x_{i_t}))$$

3. Останавливаемся, если ошибка на валидационной выборке перестает убывать

Градиентный спуск



Стохастический градиентный спуск



Стохастический градиентный спуск

1. Начальное приближение: w^0
2. Повторять, каждый раз выбирая случайный объект i_t :

$$w^t = w^{t-1} - \eta_t \nabla L(y_{i_t}, a(x_{i_t}))$$

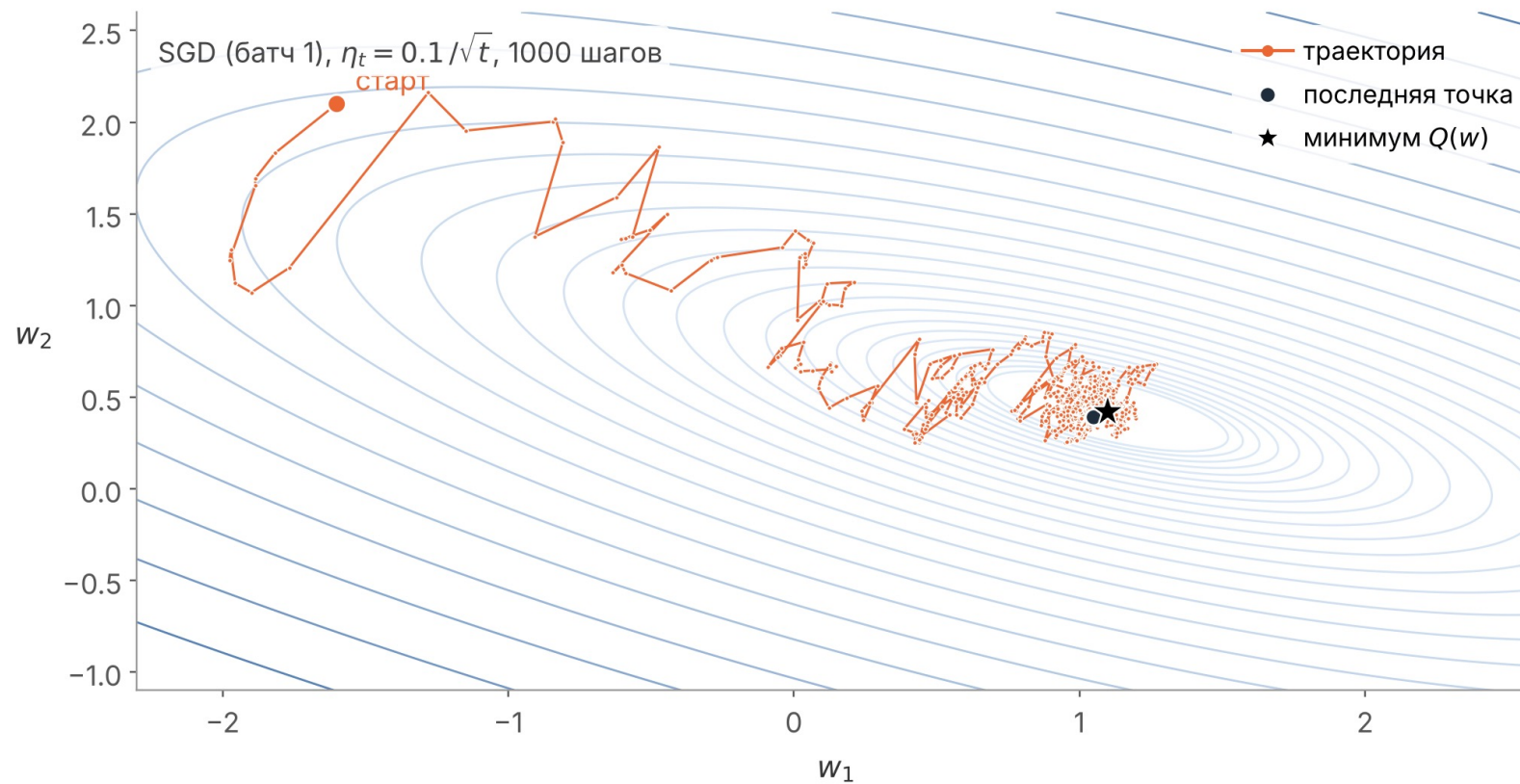
3. Останавливаемся, если ошибка на валидационной выборке перестает убывать

Стохастический градиентный спуск

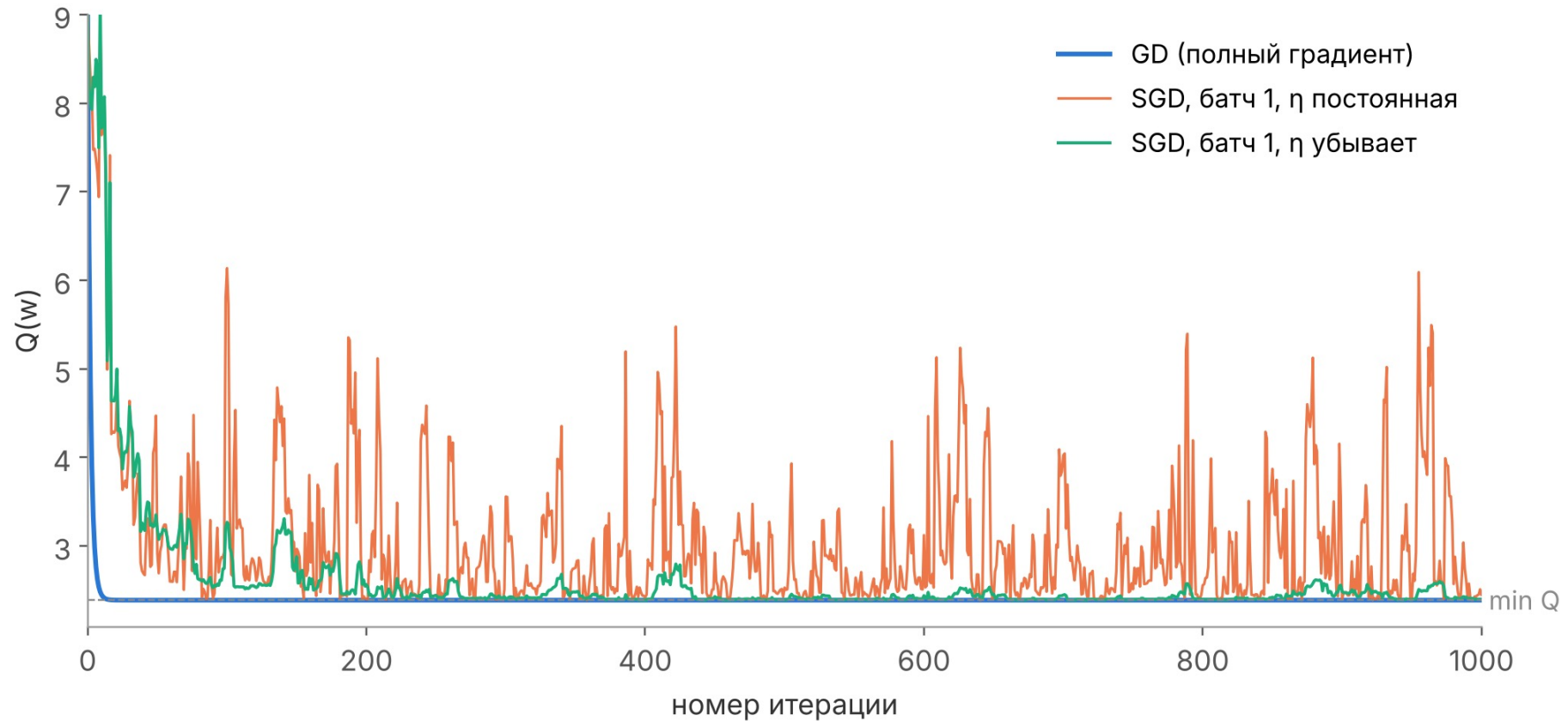
- Оценка по одному объекту **несмещённая**
- То есть в среднем мы идём в правильную сторону
- Даже в точке оптимума оценка по одному объекту вряд ли будет нулевой
- Поэтому важно, чтобы длина шага стремилась к нулю
- Сходимость к глобальному минимуму гарантируется только для выпуклых функций

Стохастический градиентный спуск

$$\eta_t = \frac{0.1}{\sqrt{t}}$$



Стохастический градиентный спуск



Mini-batch GD

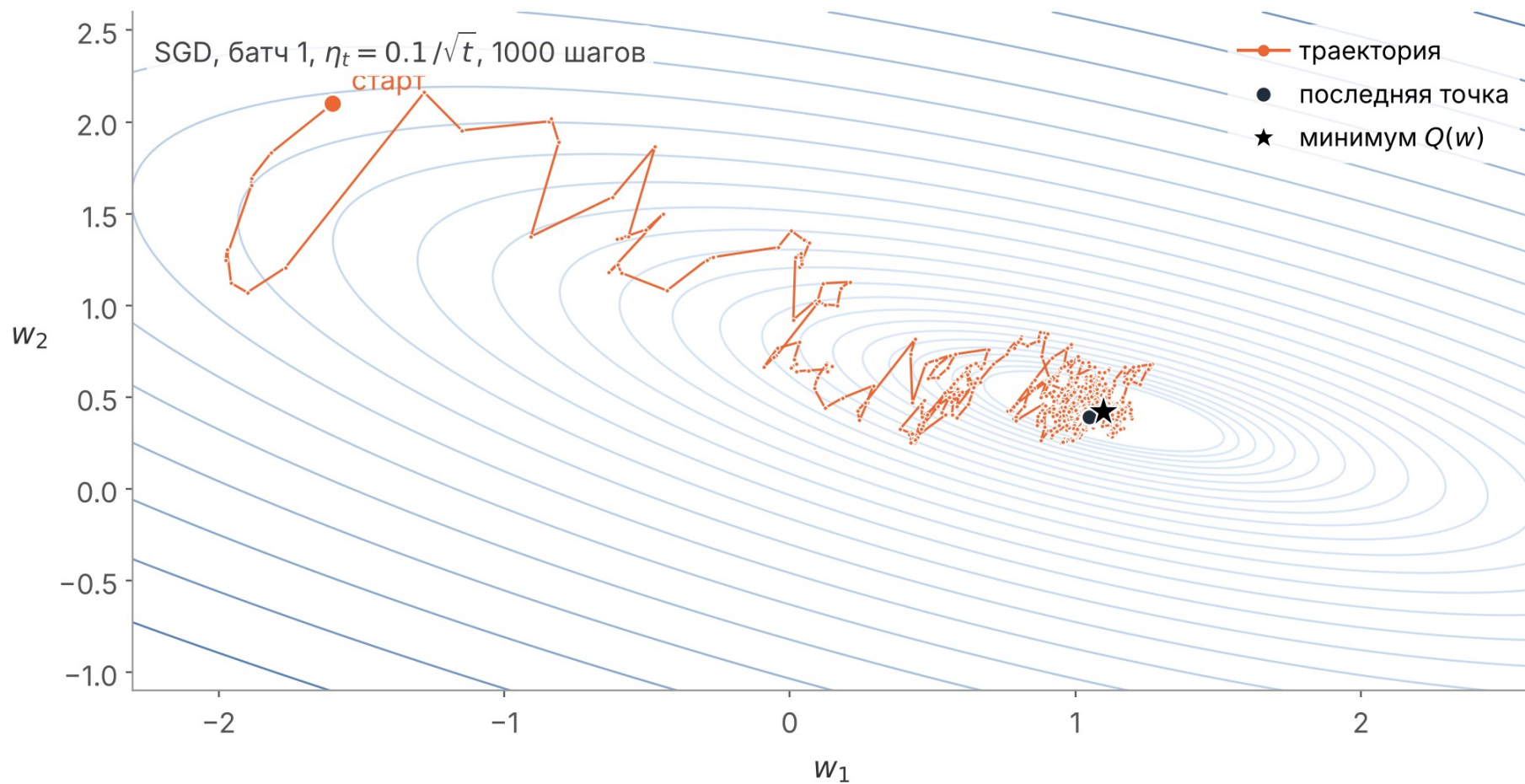
1. Начальное приближение: w^0
2. Повторять, каждый раз выбирая m случайных объектов $i_1, \dots, i_m$:

$$w^t = w^{t-1} - \eta_t \frac{1}{m} \sum_{j=1}^m \nabla L(y_{t,j}, a(x_{t,j}))$$

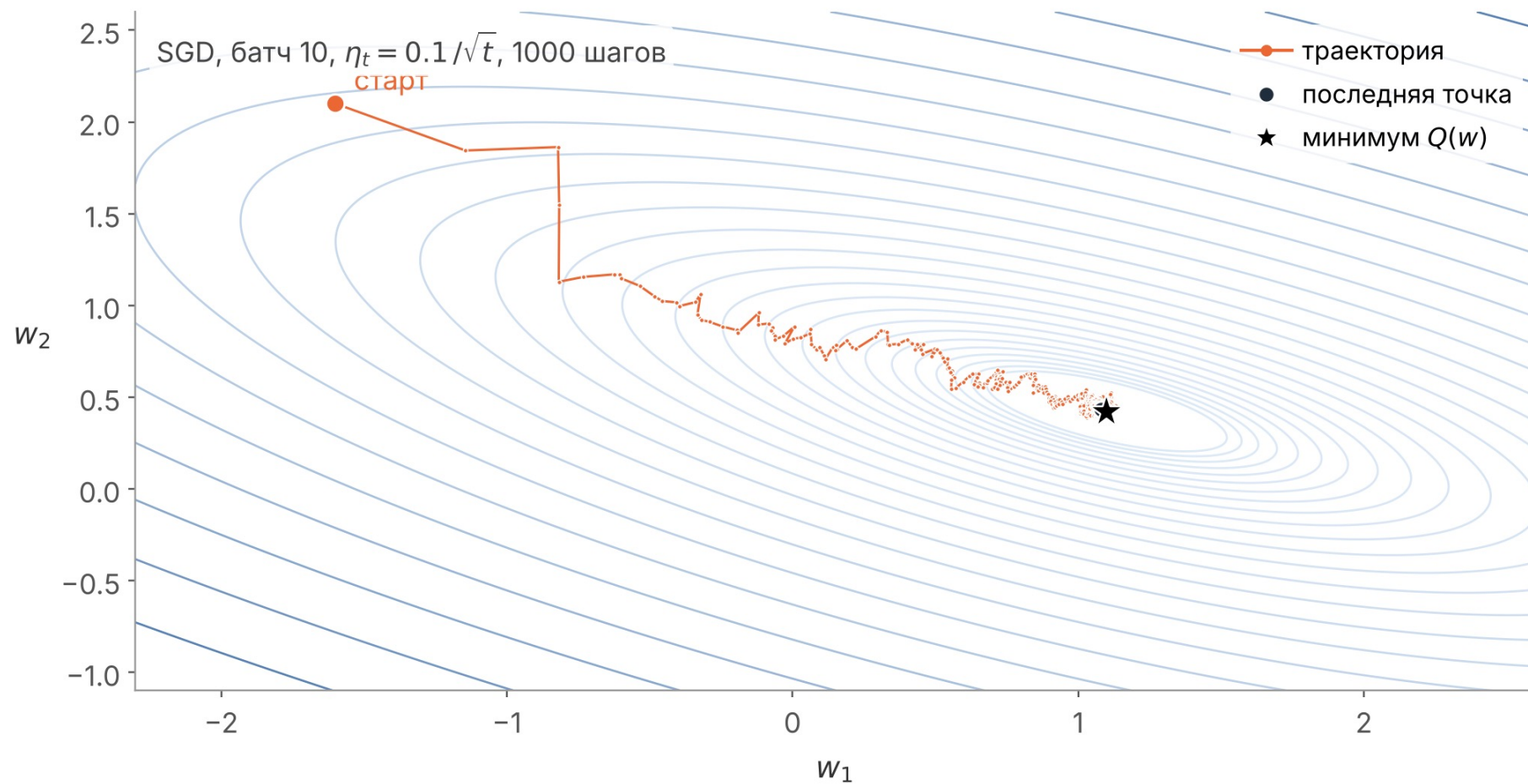
3. Останавливаемся, если ошибка на валидационной выборке перестаёт убывать

$x_{t,j}$ — объект номер j из батча, сформированного на шаге t

Батч размера 1



Батч размера 10



Batch size

- Размер батча — от десятков до тысяч
 - при обучении больших языковых моделей — миллионы токенов за шаг
- Степень двойки — скорее традиция: на современных GPU разница обычно незаметна
- Шум в оценке градиента падает как $\frac{1}{\sqrt{B}}$
 - чтобы уменьшить его вдвое, нужен батч в 4 раза больше
- Вычислительно почти так же эффективен, как шаг по градиенту одного объекта — за счёт векторизации

Batch size

The collected experimental results for the CIFAR-10, CIFAR-100 and ImageNet datasets show that increasing the mini-batch size progressively reduces the range of learning rates that provide stable convergence and acceptable test performance. On the other hand, small mini-batch sizes provide more up-to-date gradient calculations, which yields more stable and reliable training. The best performance has been consistently obtained for mini-batch sizes between $m = 2$ and $m = 32$, which contrasts with recent work advocating the use of mini-batch sizes in the thousands.

Batch size



Yann LeCun

April 27, 2018 · 🌐



Training with large minibatches is bad for your health.

More importantly, it's bad for your test error.

Friends dont let friends use minibatches larger than 32.

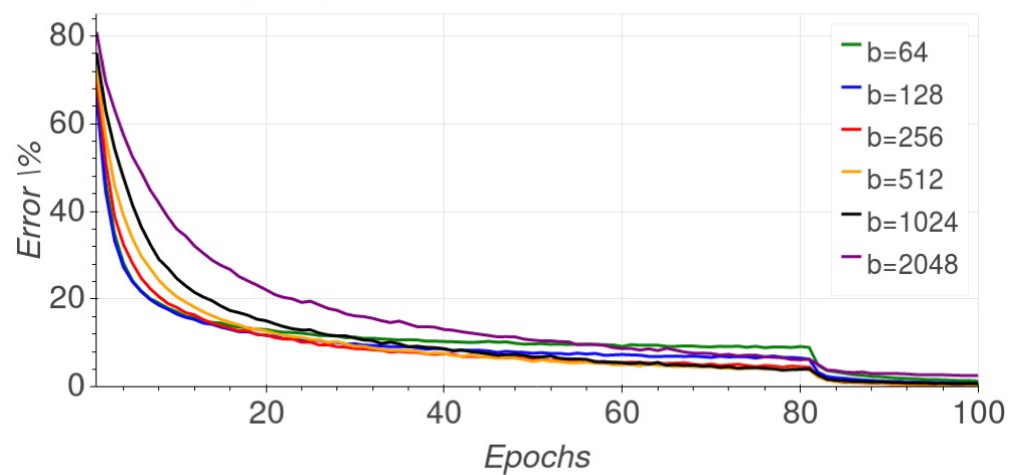
Let's face it: the **only** people have switched to minibatch sizes larger than one since 2012 is because GPUs are inefficient for batch sizes smaller than 32. That's a terrible reason. It just means our hardware sucks.

What's worse is that the easiest way to parallelize training is to make the minibatch even larger and distribute it across multiple GPUs and multiple nodes.

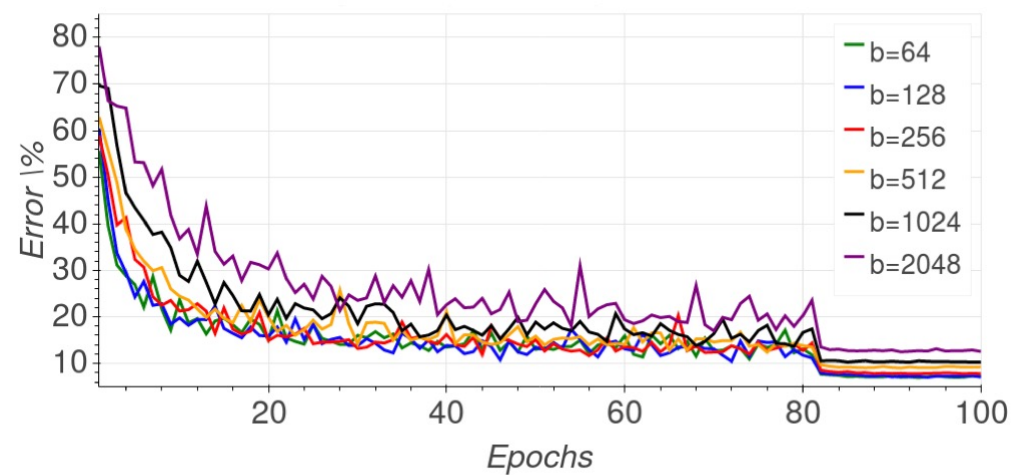
Minibatch sizes over 1024 aren't just bad for your health. They cause brain tumors.

They learn quickly, but the wrong thing.

Batch size

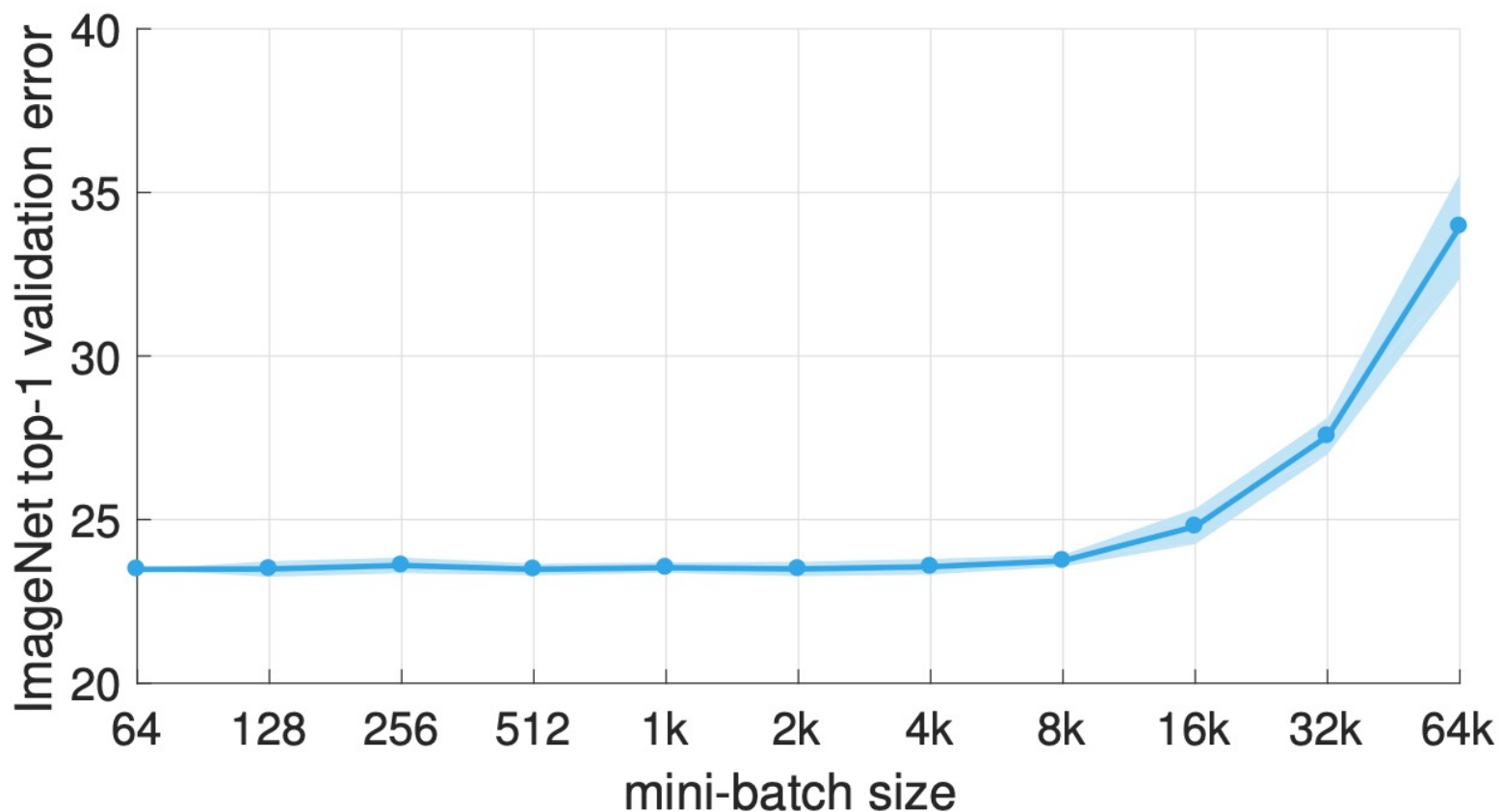


(a) Training error



(b) Validation error

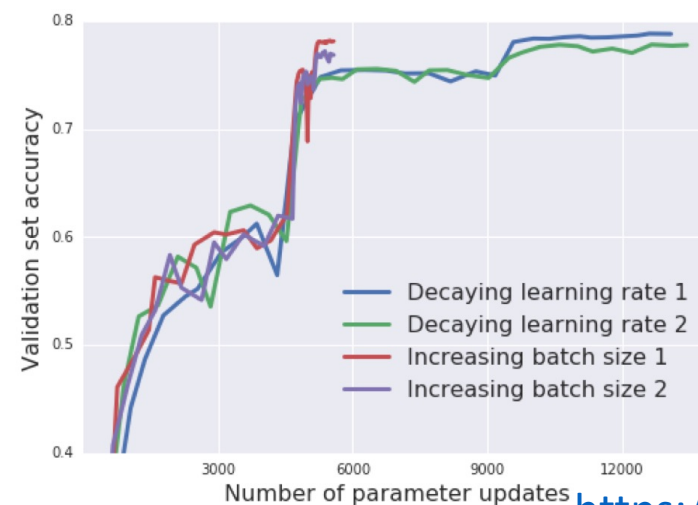
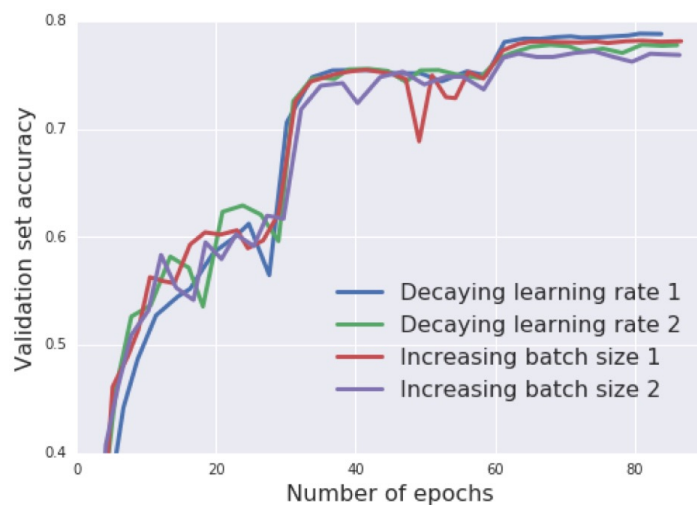
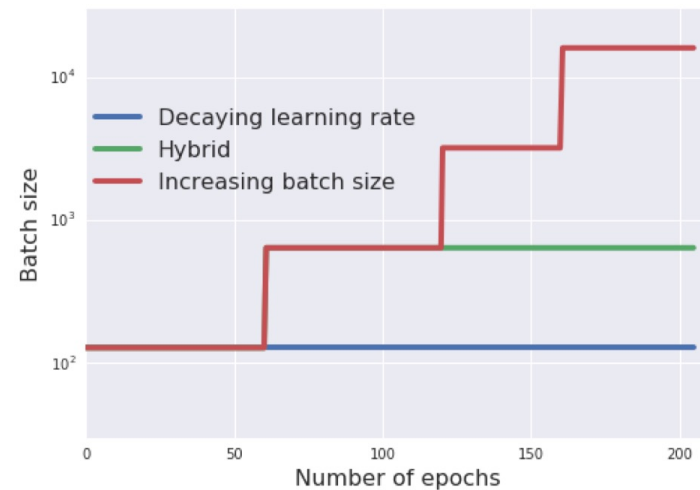
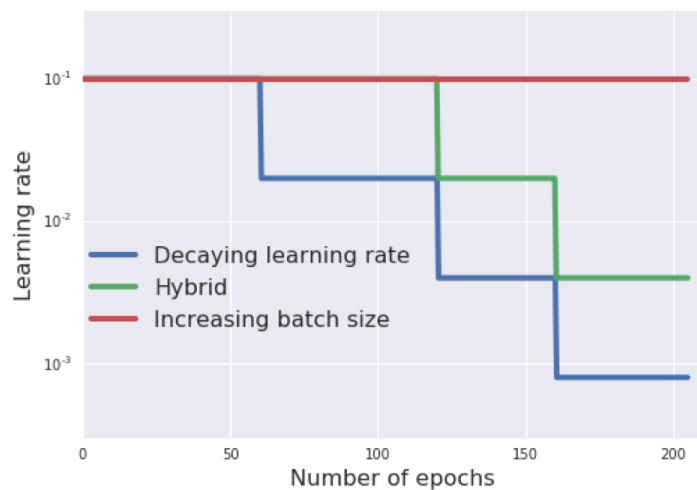
Надо грамотно подбирать формулу для длины шага!



Как подбирать длину шага для большого батча

- Linear scaling rule: увеличили батч в k раз $\rightarrow$ длину шага в k раз
- Warmup: первые несколько эпох длину шага линейно наращивают от маленькой до целевой
 - иначе большие шаги в начале обучения, пока веса случайные, разваливают модель
- Результат: обучаем ResNet-50 на ImageNet с батчем 8192 за 1 час на 256 GPU (без потери качества)
- Сейчас warmup используют почти везде, особенно при обучении трансформеров

Batch size



Критический размер батча

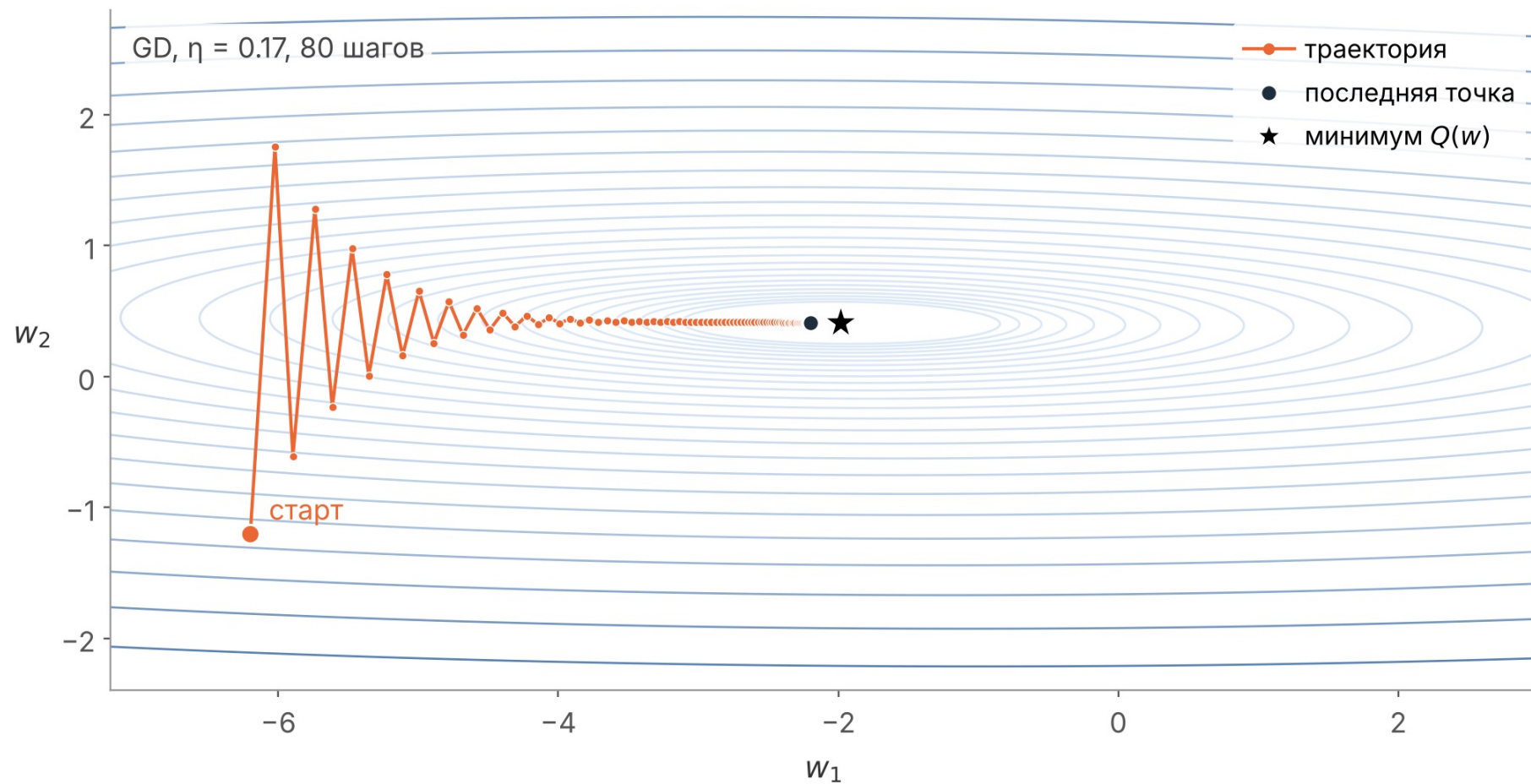
- До некоторого размера V_{crit} : увеличили батч вдвое $\rightarrow$ шагов нужно почти вдвое меньше (при правильно подобранной длине шага)
- Выше V_{crit} шагов почти не становится меньше — вычисления тратятся впустую
- V_{crit} сильно зависит от задачи: от ~ 10 (автоэнкодер на SVHN) до миллионов (Dota 2); для ImageNet — около 15 тысяч в среднем по обучению
- V_{crit} растёт по ходу обучения $\rightarrow$ батч имеет смысл увеличивать постепенно (так обучали GPT-3)

Batch size

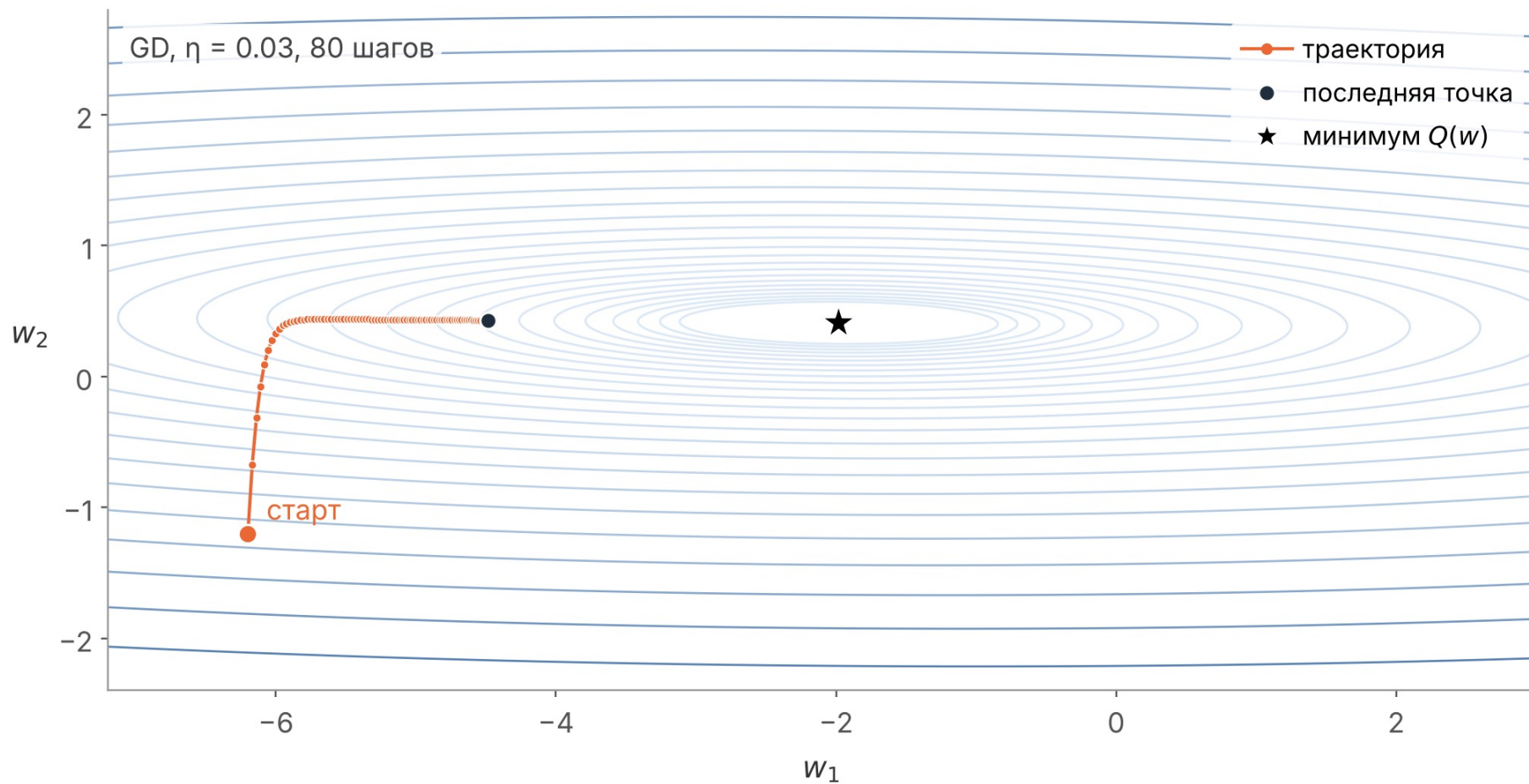
- От большого размера батча может быть польза
- Для этого надо грамотно подбирать длину шага и/или постепенно увеличить размер батча
- Польза от роста батча ограничена критическим размером

Модификации градиентного спуска

Проблемы



Проблемы



Проблемы

- Если у функции «вытянуты» линии уровня, то градиентный спуск требует аккуратного выбора длины шага и будет долго сходиться

Momentum

$$h_t = \alpha h_{t-1} + \eta_t \nabla Q(w^{t-1})$$

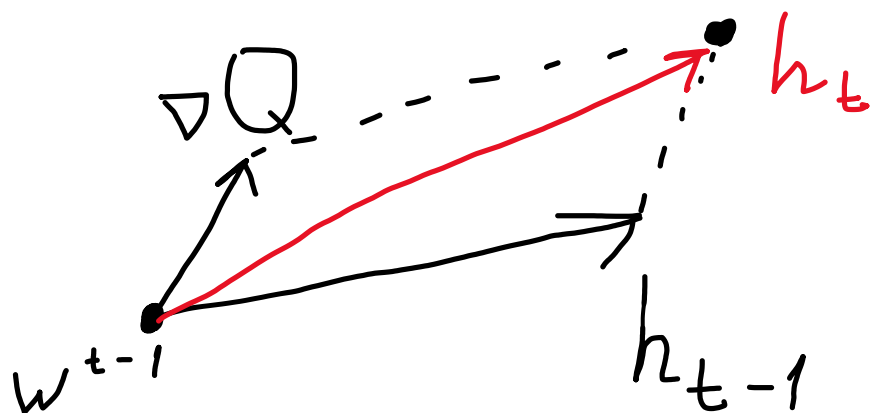
$$w^t = w^{t-1} - h_t$$

- h_t — «инерция», усреднённое направление движения
- α — параметр затухания
- Как будто шарик, который катится в сторону минимума, очень тяжёлый

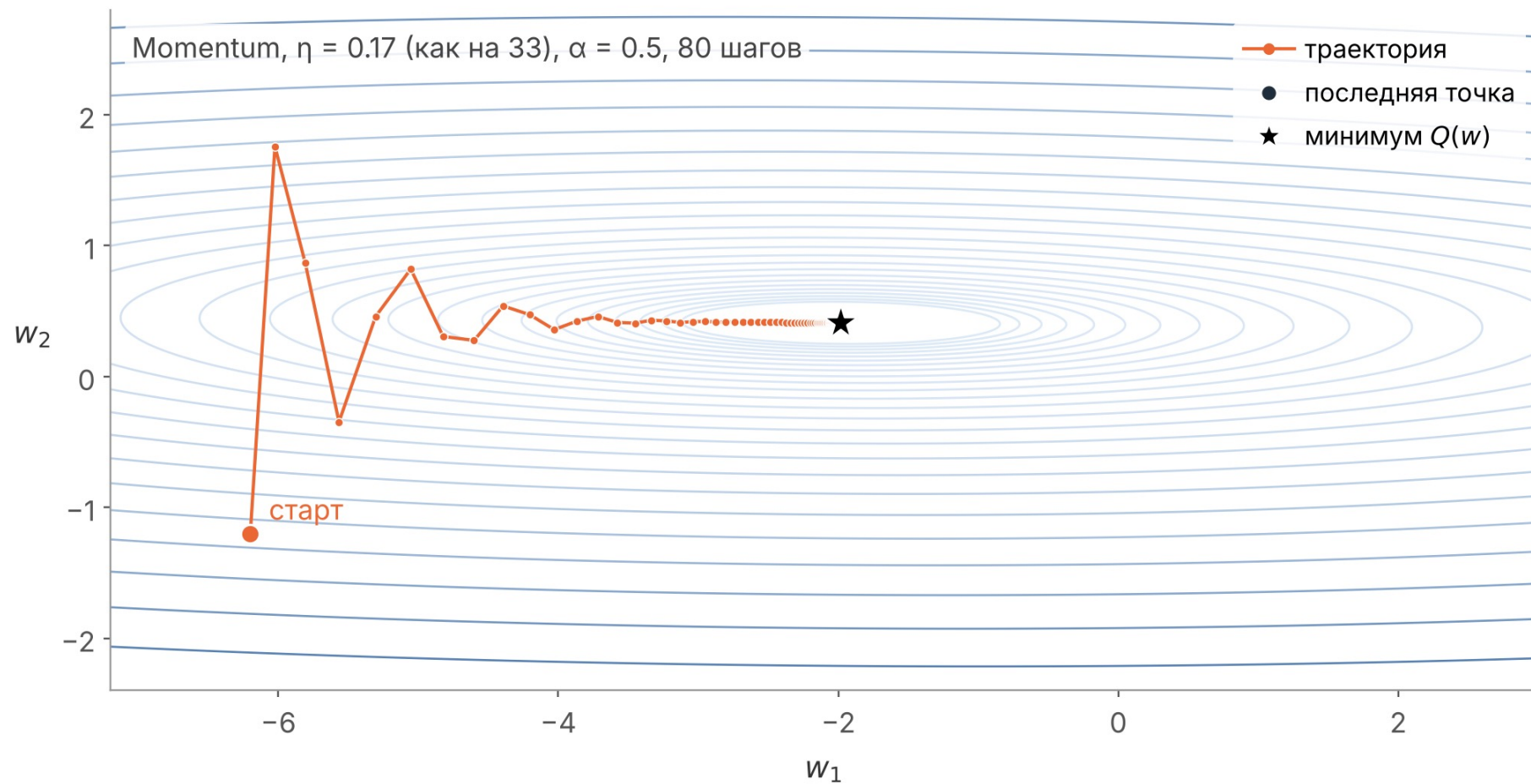
Momentum

$$h_t = \alpha h_{t-1} + \eta_t \nabla Q(w^{t-1})$$

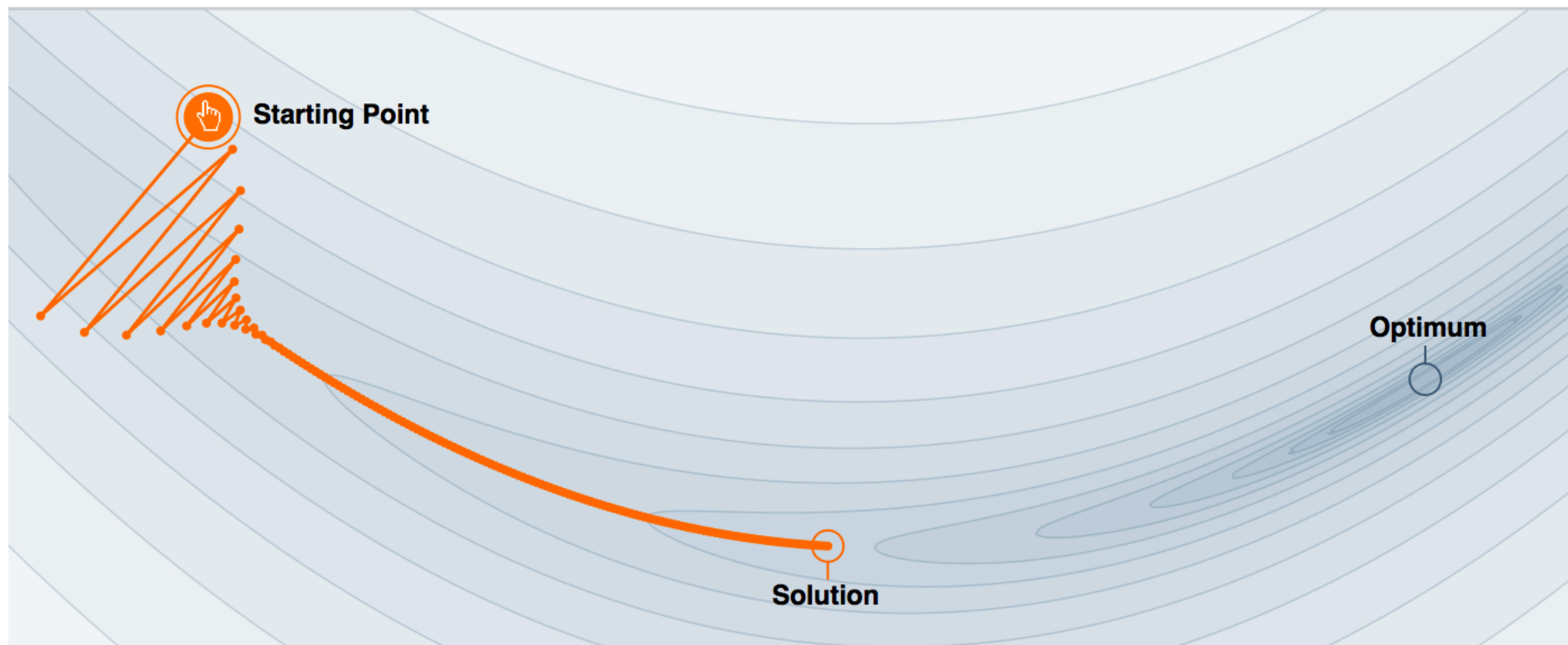
$$w^t = w^{t-1} - h_t$$



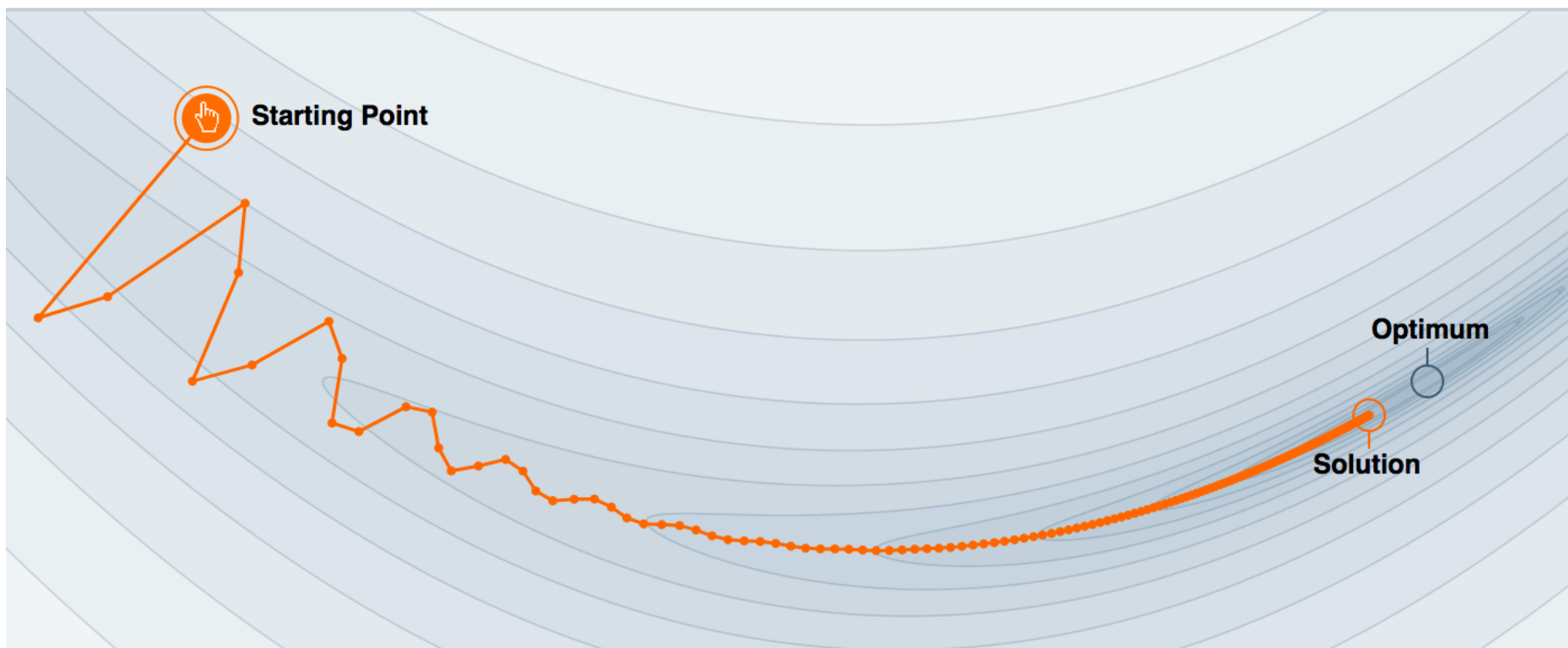
Momentum



Без инерции



С инерцией

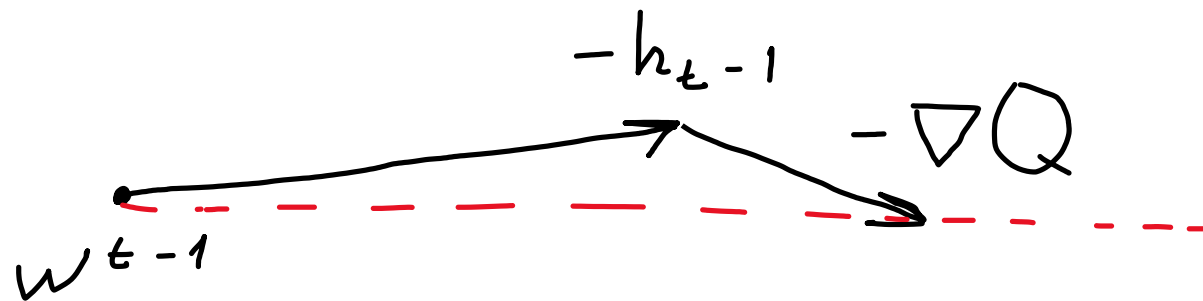
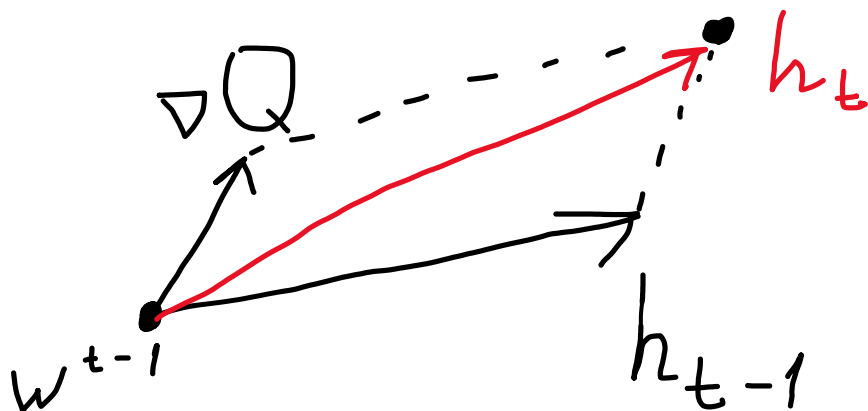


Nesterov Momentum

$$h_t = \alpha h_{t-1} + \eta_t \nabla Q(w^{t-1} - \alpha h_{t-1})$$

$$w^t = w^{t-1} - h_t$$

- $w^{t-1} - \alpha h_{t-1}$ — неплохая оценка того, куда мы попадём на следующем шаге



Проблема с разреженными данными

- Пример: модель над категориальными признаками
- Используем one-hot кодирование

	1		0
	0		0
	1		0
	1		0
	1		0
	0		0
	0		1

↑
популярная категория

↑
редкая категория

- Делаем стохастический градиентный спуск
- Через 7 шагов мы сделаем 4 обновления веса популярной категории и 1 обновление веса редкой категории

← тут уже медленные шаги

Проблема с разреженными данными

- По разным параметрам мы движемся с разной скоростью
- Будет здорово это учитывать — иначе мы обучим разные параметры с разным качеством

Проблема с разными масштабами

- Допустим, признаки имеют разный масштаб — от единиц до миллионов
- Тогда странно шагать по каждому параметру с одинаковой скоростью

AdaGrad

$$G_j^t = G_j^{t-1} + (\nabla Q(w^{t-1}))_j^2$$

$$w_j^t = w_j^{t-1} - \frac{\eta_t}{\sqrt{G_j^t + \epsilon}} (\nabla Q(w^{t-1}))_j$$

- По каждому параметру своя скорость
- η_t можно зафиксировать
- Длина шага может убывать слишком быстро и привести к ранней остановке

RMSProp

$$G_j^t = \alpha G_j^{t-1} + (1 - \alpha) (\nabla Q(w^{t-1}))_j^2$$

$$w_j^t = w_j^{t-1} - \frac{\eta_t}{\sqrt{G_j^t + \epsilon}} g_{tj}$$

- α можно взять около 0.9
- Скорость зависит только от недавних шагов

Adam

$$m_j^t = \beta_1 m_j^{t-1} + (1 - \beta_1) (\nabla Q(w^{t-1}))_j$$

$$v_j^t = \beta_2 v_j^{t-1} + (1 - \beta_2) (\nabla Q(w^{t-1}))_j^2$$

$$w_j^t = w_j^{t-1} - \frac{\eta_t}{\sqrt{\frac{v_j^t}{1 - \beta_2^t}} + \epsilon} \frac{m_j^t}{1 - \beta_1^t}$$

- Рекомендации: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Adam

- Совмещает в себе идеи из метода инерции и RMSProp

AdamW

- L2-регуляризация: добавляем к функции потерь $\frac{\lambda}{2} \|w\|^2$, в градиенте появляется слагаемое λw :

$$g_t = \nabla Q(w^{t-1}) + \lambda w^{t-1}$$

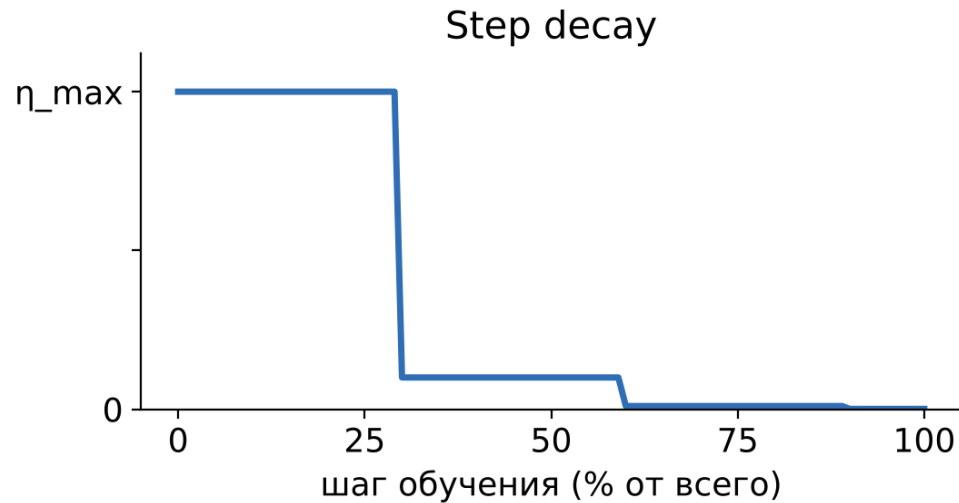
- дальше обычный шаг Adam по g_t : штраф делится на $\sqrt{\hat{v}_t}$ вместе с градиентом
 - у параметров с большими градиентами регуляризация почти исчезает
- AdamW (decoupled weight decay): штраф вычитаем из весов отдельно, минуя нормировку:

$$w^t = w^{t-1} - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda w^{t-1} \right)$$

- Для SGD обе формы совпадают, для Adam — нет

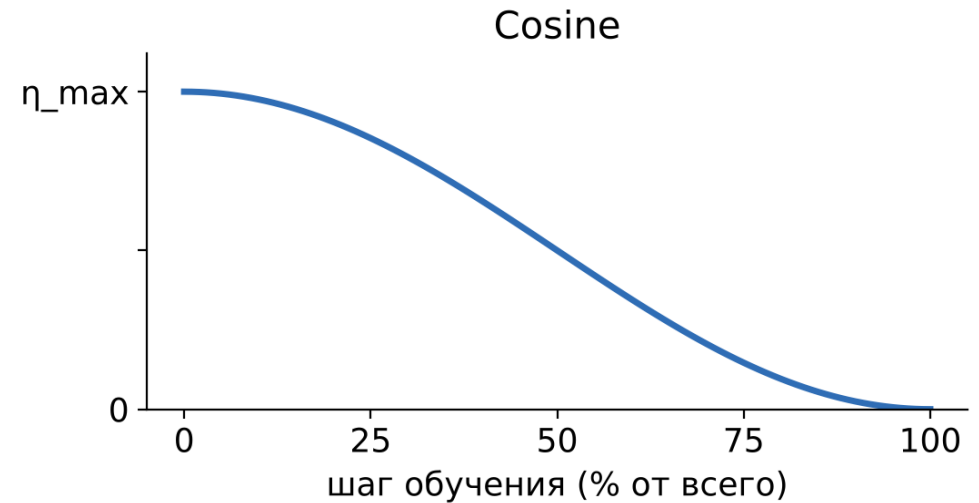
Расписание длины шага

- В начале нужен большой шаг, в конце маленький



Уменьшать в γ раз каждые T шагов. Рецепт ImageNet: 90 эпох, $\times 0.1$ каждые 30

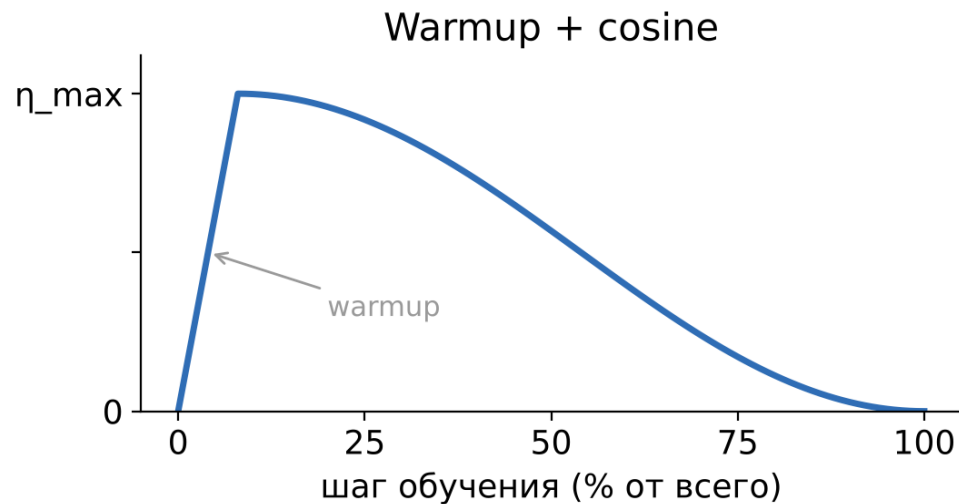
$$\eta_t = \eta_0 \cdot \gamma^{\lfloor \frac{t}{T} \rfloor}$$



Плавно от $\eta_{\max}$ почти до нуля по косинусу; T — длина обучения

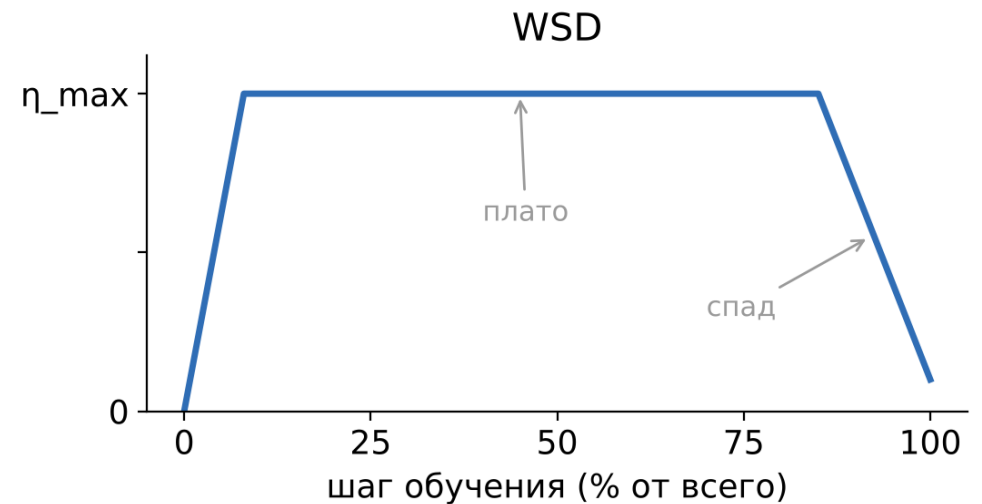
$$\eta_t = \eta_{\max} \cdot \frac{1}{2} \left(1 + \cos \left(\frac{\pi t}{T} \right) \right)$$

Warmup и WSD



Первые сотни–тысячи шагов шаг линейно растёт от 0 до η_{max} , дальше cosine. Пока веса случайные, большие шаги разваливают модель; у Adam не накоплены m и v

$$\eta_t = \eta_{max} \cdot \frac{t}{T_w} \text{ при } t \leq T_w$$



WSD = warmup, stable, decay: разогрев, длинное плато с постоянным шагом и короткий спад в последние 10–20% шагов. Удобно, когда длина обучения заранее неизвестна: с плато можно продолжить, а спад сделать в любой момент

Что используют сейчас

- Свёрточные сети: SGD + momentum 0.9, step или cosine, weight decay $1e-4$
- Трансформеры и языковые модели: AdamW, warmup + cosine, $\beta_2 = 0.95$ вместо 0.999, weight decay 0.1
- Gradient clipping — ограничить норму градиента, обычно $c = 1$:

$$g \leftarrow c \cdot \frac{g}{\|g\|} \text{ при } \|g\| > c$$

- защита от редких взрывов градиента, особенно в начале обучения

Adam или SGD?

- У SGD размер шага $\eta \cdot \|g\|$, а масштаб градиентов зависит от задачи, архитектуры, слоя и момента обучения
 - рабочий диапазон η узкий: 0.1 для ResNet, но 0.01 для другой сети уже взрывается или не учится
 - плюс надо подобрать momentum, расписание и weight decay — заново для каждой архитектуры
- У Adam шаг по каждому параметру $\approx \eta$ независимо от масштаба градиента (деление на $\sqrt{\hat{v}}$)
 - $\eta = 3 \cdot 10^{-4}$ работает почти везде, это главная причина его популярности
- Цена: вдвое больше памяти (состояния m и v на каждый параметр); на свёрточных сетях SGD иногда обобщает лучше

Muon (2024–2025)

- Веса линейного слоя — матрица W , градиент по ним G — тоже матрица. Adam масштабирует каждый элемент G по отдельности и не видит, что это матрица
- Muon берёт градиент с инерцией M_t и заменяет его на ближайшую ортогональную матрицу: все сингулярные числа становятся равны 1
$$W^t = W^{t-1} - \eta_t \cdot UV^T, \quad \text{где } M_t = U\Sigma V^T$$
 - смысл: обновление одинаково сильно двигает все направления, а не только несколько доминирующих
- Применяется к матрицам внутри сети; эмбединги, выходной слой и параметры нормализаций — по-прежнему AdamW
- Результат: до того же лосса примерно вдвое меньше вычислений, чем у AdamW (Moonshot, 2025); Kimi K2 (1 трлн параметров) обучена Muon